

Aula PL - Interface HW/SW para E/S de dados com o Raspberry Pi

Endereço base da região dos periféricos

Os registos para configuração e acesso aos pinos de GPIO estão mapeados a partir de um endereço que varia de modelo para modelo, nomeadamente:

- BCM2835 (usado no Raspberry Pi 1) - 0x20000000
- BCM2836, BCM2837 (Raspberry PI 2 e 3) - 0x3F000000
- BCM2711 (usado no Raspberry Pi model 4) - 0xFE000000

Quando o sistema computacional é baseado num sistema operativo multiprocesso (como é o caso do Windows e das distribuições Linux), os programas dos utilizadores não têm permissão para trabalhar diretamente com endereços físicos. Nestes casos, o acesso aos endereços físicos é tipicamente realizado ao nível do núcleo do sistema operativo, sendo o código de acesso aos controladores na forma de “*device drivers*”. Por sua vez, os *device drivers* disponibilizam interfaces (tipicamente na forma de ficheiros especiais) que abstraem os detalhes de acesso aos controladores de E/S.

Nesta ficha, o acesso aos endereços físicos deverá ser feito através de um ficheiro especial, `/dev/mem`, cujo conteúdo corresponde ao da memória principal. Uma vez que o acesso ao conteúdo deste ficheiro pode comprometer a estabilidade e integridade do sistema (incluindo a possibilidade apagar dados nos dispositivos de armazenamento), por omissão, apenas os processos do utilizador `root` têm permissão para aceder ao mesmo. Como tal, os programas que vai desenvolver deverão ser executados através do comando `sudo`.

Através da função `open` é possível abrir o ficheiro e através da função `mmap` é possível fazer o mapeamento das partes relevantes do ficheiro para posições de memória dos programas C (tal como ilustrado no exemplo dos *slides* da aula teórica).

Toolchain GNU Aarch64 para anfitrião x86-64

De modo a poder compilar os programas para Raspberry Pi no seu próprio computador, deverá descarregar o arquivo disponível no endereço¹

https://www.ave.dee.isep.ipp.pt/~jes/LETI-ARQCP/arm-gnu-toolchain-15.2.rel1-x86_64-aarch64-none-linux-gnu.tar.xz

e descomprimir o seu conteúdo para uma pasta do seu sistema, por exemplo para o diretório base do seu utilizador:

```
tar xvf arm-gnu-toolchain-15.2.rel1-x86_64-aarch64-none-linux-gnu.tar.xz -C ~/
```

Este ficheiro disponibiliza uma *toolchain* (compilador, *linker*, bibliotecas, etc.) que permite fazer a compilação dos programas em C para executáveis GNU/Linux para a arquitetura Aarch64. O nome do executável do compilador é `aarch64-none-linux-gnu-gcc`. Para evitar a necessidade de indicar o caminho para o compilador em cada execução, pode criar uma ligação para o mesmo num dos diretórios que fazem parte da variável `PATH`, por exemplo:

¹ Descarregado originalmente de <https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads>

```
mkdir -p ~/bin  
ln -s ~/arm-gnu-toolchain-15.2.rel1-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-  
linux-gnu-gcc ~/bin/arm-gcc
```

Desta forma, para compilar os seus programas para a arquitetura aarch64, basta executar o comando **arm-gcc** com os parâmetros habituais do **gcc**.

Exercício 1 – Configuração de pinos como saídas digitais

As linhas de GPIO são configuradas através de um conjunto de 24 endereços, situados na posição **0x200000** relativamente ao endereço base da região dos periféricos.

Cada 32 bits permitem configurar 10 pinos de GPIO, sendo que a configuração de cada GPIO é efetuada através de 3 bits, tal como ilustrado na tabela abaixo:

	Endereço relativo												
Bit	0x200000	0x200004	...										
0	GPIO 0	GPIO10											
1													
2													
3	GPIO 1												
4													
5													
6	GPIO 2												
7													
8													
9	GPIO 3												
10													
11													
12	GPIO 4												
13													
14													
15	GPIO 5												
16													
17													
18	GPIO 6												
19													
20													
21	GPIO 7												
22													
23													
24	GPIO 8												
25													
26													
27	GPIO 9												
28													
29													
30													
31													

Para configurar um pino como saída, deverá escrever o valor 1 nos respetivos bits de controlo (**001** em binário). Para configurar como entrada o valor é 0. Os restantes 6 valores permitem selecionar funções alternativas.

O mecanismo para estabelecer o valor de um pino configurado como saída digital não é o mais óbvio: existe um conjunto de endereços que devem ser usados para colocar as saídas a 1 (SET) e outro conjunto de endereços que devem ser usados para colocar as saídas a 0 (CLEAR). Mais especificamente:

- Os registos para colocar as saídas a 1 (SET) estão mapeados nos endereços 0x001C a 0x0023 relativos ao endereço base do GPIO. Cada bit da sequência de endereços corresponde a um pino GPIO, também em sequência (bit 0 do primeiro endereço para GPIO 0, etc.). Para colocar um pino no nível lógico alto, deverá colocar o bit correspondente a 1;
- Os registos para colocar as saídas a 0 (CLEAR) estão mapeados nos endereços 0x0028 a 0x002F relativos ao endereço base do GPIO, correspondendo cada bit da sequência a um dos pinos GPIO, de forma análoga aos registos de SET. Para colocar um pino no nível lógico baixo, deverá colocar o bit correspondente a 1.

A seguinte definição permite fazer o mapeamento entre os diferentes registos do módulo

de GPIO e uma variável da linguagem C:

```
typedef struct {
    uint32_t GPFSEL[7]; /*!< GPIO Function Select */
    uint32_t GPSET[3]; /*!< GPIO Pin Output Set */
    uint32_t GPCLR[3]; /*!< GPIO Pin Output Clear */
    uint32_t GPLEV[3]; /*!< GPIO Pin Level */
    uint32_t GPEDS[3]; /*!< GPIO Pin Event Detect Status */
    uint32_t GPREN[3]; /*!< GPIO Pin Rising Edge Detect Enable */
    uint32_t GPFEN[3]; /*!< GPIO Pin Falling Edge Detect Enable */
    uint32_t GPHEN[3]; /*!< GPIO Pin High Detect Enable */
    uint32_t GPLEN[3]; /*!< GPIO Pin Low Detect Enable */
    uint32_t GPAREN[3]; /*!< GPIO Pin Async. Rising Edge Detect */
    uint32_t GPAFEN[3]; /*!< GPIO Pin Async. Falling Edge Detect */
    uint32_t GPPUD; /*!< GPIO Pin Pull-up/down Enable */
    uint32_t GPPUDCLK[3]; /*!< GPIO Pin Pull-up/down Enable Clock */
} bcm2837_gpio_registers_t;
```

Escreva um programa que, através do acesso direto aos endereços físicos do processador, permita ativar e desativar periodicamente (fazer “piscar”) um LED ligado ao pino correspondente à linha GPIO 18².

Ensaio no Raspberry Pi

Após gerar o executável, deverá copiá-lo para o Raspberry Pi e executá-lo remotamente. Essa operação pode ser feita através dos comandos `ssh`³ e `scp`, como se explica de seguida.

Abra um novo terminal e execute o seguinte comando:

```
ssh jes@10.0.14.XXX
```

onde o XXX se refere ao dígito que distingue o último octecto dos endereços, isto é, os diferentes endereços IP do Raspberry Pi.

Após estabelecer a ligação SSH ao Raspberry Pi, deverá criar um novo diretório cujo nome deverá ser o seu número de estudante. Por exemplo, para um estudante com o número 1220001:

```
mkdir 122001
```

Finalmente, para copiar o executável, deverá utilizar o seguinte comando (a partir de um terminal do computador onde está a trabalhar):

```
scp execfile ubuntu@10.0.14.12X:student\_number/
```

onde `student_number` deverá ser o seu número de estudante.

² Os pinos de GPIO do Raspberry Pi não deverão ser forçados a fornecer continuamente mais do que 3 mA, sob pena de danificar o sistema. Tendo em conta a curva característica de queda de tensão em função da corrente típica do LED, deverá utilizar uma resistência com um valor de cerca de 1 k Ω em série com o LED. Por sua vez estes, componentes deverão estar ligados entre o pino de GPIO e um pino de GROUND.

³ O comando `ssh` recorre ao Secure Shell Protocol (SSH) para estabelecer uma ligação segura com um computador remoto e, através desta, executar comandos remotamente usando a consola local como dispositivo de entrada e saída de dados.

Exercício 2 – Configuração de um modulador de largura de impulsos

De uma forma geral, um PWM permite escolher a duração de um impulso para cada período de um sinal de referência (a frequência do PWM). A duração do impulso poderá variar entre 0 e a duração do período do sinal de referência.

Os módulos de PWM do BCM2711 baseiam-se no seguinte modo de funcionamento:

- Escolha de uma fonte de sinal periódico para definição da frequência de referência. O Raspberry Pi 4 usa uma frequência base de 54 MHz (as versões anteriores usavam uma frequência de 19,2 MHz).
- Escolha da frequência do “relógio do PWM”, que resulta da divisão da frequência de referência por um valor inteiro entre 2 e 4095 (**divisor**).
- Escolha da resolução da largura dos impulsos, ou seja, o número de níveis **N** possíveis para a duração do impulso em cada período do sinal de PWM. Em funcionamento normal, quando se solicita um impulso de largura **C**, a saída ficará a 1 durante **C** períodos do sinal de relógio do PWM e a 0 durante **N-1-C** períodos desse mesmo sinal⁴. Deste modo, pode concluir-se que quanto maior for o número de níveis possíveis menor será a frequência “portadora” resultante, i.e., a frequência do PWM. Nomeadamente, o valor desta frequência é dado por:

$$\text{Frequência do PWM} = 54000000/\text{divisor}/(N-1) \text{ Hz}$$

Os registos do primeiro módulo de PWM estão mapeados a partir da posição 0x20C000 relativamente ao endereço base da região dos periféricos. Tal como no exercício anterior, a organização destes registos na memória pode ser mapeada para um tipo de dados da linguagem C:

```
typedef struct {
    uint32_t CONTROL;
    uint32_t STATUS;
    uint32_t DMAC[2];
    uint32_t CHN0_RANGE; //maximum value for PWM_CHANNEL0_DATA
    uint32_t CHN0_DATA; //Pulse Width for CHANNEL0 (from 0 to PWM_CHN0_RANGE)
    uint32_t FIF1[2]; // PWM FIFO Input
    uint32_t CHN1_RANGE;
    uint32_t CHN1_DATA;
} bmc2711_pwm_registers_t;
```

Escreva um programa que, recorrendo ao canal 0 do PWM0 do BCM2711 e mantendo os valores por omissão para a frequência do relógio do PWM, permita controlar a tensão média no pino GPIO 12 numa escala linear de 256 níveis, onde 0 corresponde a um sinal sempre a 0 e 255 corresponde a um sinal contínuo a 1.

Após o arranque, o programa deverá realizar os seguintes procedimentos:

- Carregar para **CHN0_RANGE** o valor correspondente ao sinal contínuo a 1 (255, neste caso);
- Colocar a 1 o bit 0 (**PWEN1**) do registo **CONTROL**, para ativar o PWM0.

O valor desejado para a largura dos impulsos, na escala de 0 a 255, deverá ser carregado para o registo **CHN0_DATA**.

Teste o programa implementado. Se ligar um LED ao pino do conector J8 correspondente ao GPIO 12, tendo em conta os cuidados indicados no exercício anterior, poderá observar a variação do nível de luminosidade do mesmo de acordo com a largura dos impulsos.

⁴ Note que para **N** níveis possíveis, a gama de valores possíveis varia entre 0 e **N-1**.